

PROJECT SUMMARY

BrightPath AI Sales Assistant

A lead arrives, is captured, analysed, scored against an explicit written rubric, given a drafted follow-up and one recommended next action — and tracked through a status a human still owns.

SUBMITTED BY

Lordmark Ebimobowei Dorgu

STATUS

Live · 50 checks green

DATE

25 August 2026

LIVE APPLICATION

[brightpath-dashboard.vercel.app](#)

THE PROBLEM BRIGHTPATH BROUGHT

Nobody answers fast enough

Every lead is somebody's job; no lead is anybody's deadline. Cold ones go cold quietly.

Good leads look like the rest

A funded buyer with a deadline arrives in the same inbox, in the same font, as a price enquiry.

Reps re-read enquiries by hand

Pulling size, industry, budget, need and intent out of prose is slow and inconsistent.

Follow-ups written from scratch

The task that gets postponed — usually on exactly the leads that matter most.

THE JOURNEY, COMPLETED BEFORE A REP OPENS THE RECORD

1

Lead arrives

2

Captured & normalised

3

Evidence extracted

4

Scored & prioritised

5

Follow-up drafted

6

Next action recommended

7

Status tracked by a human

THE DESIGN DECISION IT RESTS ON

The model extracts evidence. Deterministic code computes the score.

Every fact the analyst reports carries the **verbatim quote it came from**, and a versioned rubric — **brightpath-bant-1.0.0**, in source control — applies BrightPath's policy to those facts. The model is never told what a good lead looks like, because telling it would invite it to flatter whichever lead it is reading. So the same evidence scores identically on every run, every point traces to a named rubric line, and a score is **withheld rather than guessed** when the evidence is too thin.

RUNNING NOW, READ FROM THE LIVE PIPELINE

15

Leads in pipeline
10 marked high priority

13

Carry a published score
2 withheld for human review

6m

Median first useful action
Across 7 touched leads

50

Automated checks green
Scoring 18 · failover 12 · journey 20

WHAT WAS BUILT

- **Seven modules** — intake, analyst, scoring, follow-up writer, next-action advisor, status & activity, and an SLA clock.
- **A full REST API** at `/api/v1` — all ten endpoints the brief requires, plus source webhooks, stats and a health probe.
- **Five working screens** — pipeline, triage, capture, the lead record and an inbox, all on live data.
- **Real email**, marked `sent` only against the message id the mail server returns — no button just flips the column.
- **An append-only timeline** naming who made each change — `rep`, `ai:analyst`, `ai:writer`.

BUILT WITH

- **Claude Code** as the primary development environment — architecture, implementation, verification and documentation.
- **Visual Studio Code** as the editor and terminal host driving it.
- **Next.js 16** (App Router, React 19), **TypeScript 5.9**, **Tailwind 4** and **shadcn/ui** — one app holding both API and interface.
- **Neon** Postgres with **Drizzle ORM**; **Zod** contracts shared by routes and pages, so a filter cannot mean two things.
- **Gemini** primary, failing over through **Groq** and **OpenRouter** to an honest stub — on any failure, not only rate limits.
- Deployed on **Vercel** from **GitHub** on every push to `master`.

Stated plainly rather than left to be found. Authentication is one shared token, not user accounts — tenant scoping is real and proven by four cross-tenant checks, but the sign-in screen is a presentation gate that checks no password. Rate-limit counters are per-instance, webhooks verify no signature, and mail is send-only. Tasks, Calendar, Team, Pricing and FAQs are interface-template scaffolding, not part of this submission.